HyperPot

Fault-Tolerant Distributed LLM Inference on a Pool of Consumer Laptops

A Simplified, Evidence-Tagged Idea & Build Guide

HyperPot — Meta Global AI Developer Hackathon (Indie Track)

Version 1.0

Abstract

This document explains HyperPot in plain language: what it is, the one problem it solves, and exactly how it will be built. HyperPot pools ordinary student laptops — across different GPUs and operating systems, connected over Wi-Fi — into a single cluster that runs a Meta Llama model. Its defining capability is **fault tolerance**: when one laptop drops off the network mid-generation (lid closed, Wi-Fi lost, battery dead), the system detects it, re-routes that laptop's share of the work to a surviving machine, and the answer keeps streaming. Every important claim here is tagged **[VERIFIED]** (verified from a source), **[HYPOTHESIS]** (a reasoned hypothesis), or **[UNKNOWN]** (unknown / unproven), so the reader knows exactly what to trust. The guide is honest about novelty: the *algorithm* of load-aware failure recovery already exists in datacenter research; HyperPot's contribution is bringing it down to a pile of cheap, unreliable laptops on Wi-Fi using only free tools — and making it demonstrable live. The final chapters give the technology stack, the hardware map, a phase-by-phase MVP plan to the submission deadline, measurable benchmarks, and the hackathon strategy.

Contents

1 How to Read This Guide **3**
- 1.1 The evidence tags . . . 3
- 1.2 Keep vs drop . . . 3
- 1.3 The one-paragraph mental model . . . 3
- 1.4 The honest novelty statement . . . 3

2 The Idea **3**
- 2.1 What HyperPot is . . . 3
- 2.2 The original architecture (the starting point) . . . 3
- 2.3 The defining problem it solves . . . 4
- 2.4 What this guide deliberately is NOT about . . . 4

3 The Competitive Landscape (Verified) **4**
- 3.1 The existing projects . . . 4
- 3.2 The key verified facts . . . 5
- 3.3 What this means for novelty . . . 5

4 What Fault Tolerance Solves (In Plain Words) **5**
- 4.1 The group-essay analogy . . . 5
- 4.2 The LLM version . . . 5
- 4.3 A real-time example . . . 5
- 4.4 The improved recovery idea (capability-aware) . . . 6
- 4.5 What it solves, in one line . . . 6

5 The Technology Stack **6**
5.1 Component by component . . . 6
5.2 Summary table . . . 7

6 The Hardware **7**
6.1 The five laptops (verified with the owner) . . . 7
6.2 Why this mix helps . . . 7
6.3 The honest warning . . . 8
6.4 Recommended configuration . . . 8

7 How It Works (Architecture) **8**
7.1 The normal path . . . 8
7.2 The failover path (the novel core) . . . 8
7.3 Text diagram . . . 8
7.4 Why the controller, not llama.cpp, owns recovery . . . 9

8 The MVP Plan (Phases) **9**
8.1 Scope discipline . . . 9
8.2 The must-do verification before committing (de-risking) . . . 9

9 Benchmarks (Measurable & Honest) **10**
9.1 Primary metrics (the differentiator) . . . 10
9.2 Secondary metrics (context only — where we do NOT win) . . . 10
9.3 Baselines to run yourself . . . 10

10 Hackathon Strategy (Meta Global AI Developer Hackathon) **10**
10.1 The verified facts about the event . . . 10
10.2 Why Indie fits HyperPot . . . 10
10.3 The pitch (safe wording) . . . 11
10.4 Meta-model requirement . . . 11
10.5 Demo safety . . . 11

11 Risks & Immediate Next Steps **11**
11.1 The honest risks . . . 11
11.2 Guardrails being held . . . 11
11.3 Immediate next steps . . . 11
11.4 Status of this document . . . 11

12 Evidence Log: Kill-Test on Real Hardware **12**
12.1 Setup . . . 12
12.2 Run 1 — normal distributed inference (baseline) . . . 12
12.3 Run 2 — remote node lost (the premise) . . . 12
12.4 What this proves . . . 12
12.5 Note for the live demo . . . 12

13 References & Sources **13**
13.1 Research papers (competitive landscape) . . . 13
13.2 Official repositories & documentation . . . 14
13.3 Hackathon (rules & requirements) . . . 15
13.4 Evidence-strength note . . . 15

1 How to Read This Guide

HyperPot turns a collection of ordinary laptops into one shared computer that runs a large language model (LLM). This guide documents the *idea*, the *evidence* behind it, and the exact *plan* to build it for the Meta Global AI Developer Hackathon (Indie track). It is written so a newcomer can follow it, while remaining strictly honest about what is proven and what is not.

1.1 The evidence tags

Next to important claims you will see one of three tags. They answer the question "how much should I trust this?":

- **[VERIFIED]** — a **verified fact** taken from an official repository, research paper, or the hackathon's own rules. These can be stated to a judge.
- **[HYPOTHESIS]** — a **reasoned hypothesis**: a sensible inference that has not yet been proven by a test or a source.
- **[UNKNOWN]** — **unknown**: not enough evidence yet; must be checked before relying on it.

1.2 Keep vs drop

Design decisions are marked **KEEP** (part of the plan) or **DROP** (deliberately excluded, with a reason). Being explicit about what we are *not* building is as important as what we are.

1.3 The one-paragraph mental model

A single LLM is split into layers. Each laptop (a "worker") loads a slice of those layers. To answer a prompt, data flows worker 1 → worker 2 → worker 3 like an assembly line, and words come out the end. A central "controller" hands out the work, listens for heartbeats, and — the key part — when a worker goes silent, it reassigns that worker's slice to a healthy machine and re-runs the request so the answer continues. The whole point is that the laptops are *unreliable by nature* (they are people's real machines on Wi-Fi), and HyperPot makes the cluster behave reliably anyway.

1.4 The honest novelty statement

We do not claim to have invented load-aware failure recovery for LLMs. That already exists in datacenter research (§3). HyperPot's contribution, stated safely, is: *"Fault tolerance like this exists in datacenters. It has never run on a pile of cheap student laptops over Wi-Fi with free tools. We made that work — and here, watch, we kill one."* This framing is defensible because every part of it is backed by a source in §3.

2 The Idea

2.1 What HyperPot is

HyperPot is a distributed LLM system that uses **consumer laptops as shared computing resources**. Instead of renting a cloud GPU or needing one expensive machine, several everyday laptops pool their compute and memory to run a model together.

2.2 The original architecture (the starting point)

The project begins from an existing three-part design:

- **Go** — the master controller: HTTP API, task queue, worker heartbeats, and result collection. **KEEP**
- **Rust** — the worker agents that connect to the controller and execute inference tasks. **KEEP**
- **llama.cpp** — the LLM inference engine (originally prebuilt Windows binaries). **KEEP**, but used as a *local* engine only (see §5).

2.3 The defining problem it solves

The laptops belong to real people and sit on Wi-Fi, so they come and go: a lid closes, Wi-Fi drops, a battery dies. HyperPot's defining capability is to **survive that** — to keep generating the answer even when a laptop vanishes mid-sentence. This is the one thing the project is built around.

2.4 What this guide deliberately is NOT about

- **DROP Distributed fine-tuning / training.** It overlaps existing projects (Petals, QVAC Fabric), is far harder to make fault-tolerant, and adds nothing to the live demo. HyperPot is **inference only**. If personalization is wanted, a small LoRA adapter can be trained offline on one machine beforehand — never distributed training during the event. **[HYPOTHESIS]**
- **DROP Beating anyone on raw speed.** Faster heterogeneous inference is already owned by prima.cpp (§3). We do not compete there.

3 The Competitive Landscape (Verified)

This chapter is the evidence base. Everything here was checked against official repositories, papers, or issue trackers.

3.1 The existing projects

Table 1: What existing distributed-inference projects actually do

Project	What it does	Fault tolerance on node loss
llama.cpp RPC	Offloads layers to remote `rpc-server` nodes over TCP	**None** — a peer crash/drop "terminates the process" **[VERIFIED]**
Exo	Auto-discovery cluster, best on Apple Silicon (MLX, Thunderbolt)	"Critical gaps in fault tolerance" per 2025 review + open issues **[VERIFIED]**
Petals	BitTorrent-style swarm for 50B+ models over the Internet; also fine-tuning	Designed for churn, but Internet-scale / large models (not LAN laptops) **[VERIFIED]**
Distributed Llama	Home-device tensor parallel over Ethernet	Static topology; no churn-recovery claim **[VERIFIED]**
prima.cpp	30–70B on heterogeneous home clusters; Halda scheduler; Wi-Fi tolerant	Static, up-front scheduling; no mid-inference recovery emphasis **[VERIFIED]**
QVAC Fabric	Edge-first inference + LoRA fine-tuning on phones/laptops/GPUs	Marketing claims decentralization; internals **[UNKNOWN]**

3.2 The key verified facts

- **[VERIFIED] llama.cpp RPC has no fault tolerance.** Its own docs state a peer crash, a dropped network, or a malformed response terminates the process (via `RPC_STATUS_ASSERT/GGML_ABORT`). Confirmed by GitHub issues #11508 and #9214.
- **[VERIFIED] Exo's fault tolerance is weak in practice.** A Sept 2025 deep-dive review cites "critical gaps in security, fault tolerance, and operational tooling"; open issues show nodes dropping and unstable node counts.
- **[VERIFIED] Petals already solved churn-tolerant inference** — but over the *Internet* for *50B+* models, not a LAN laptop pool.
- **[VERIFIED] Capability-aware failure recovery already exists in research**: LUMEN (arXiv 2606.17787) does "interrupted-request distribution at failure time" considering cluster load; "Intelligent Orchestration ... at the Edge" (arXiv 2504.03668) does capacity-aware distribution + dynamic partition migration; FailSafe (arXiv 2511.14116) survives multiple GPU failures. These target **GPU/cloud/edge-server** clusters, not consumer laptops on Wi-Fi with llama.cpp.

3.3 What this means for novelty

- As a *research algorithm*, load-aware recovery is **not novel**. **[VERIFIED]** Do not claim invention to a judge.
- As a *working system* on consumer laptops, over Wi-Fi, using llama.cpp (which cannot do it) — that combination is **unclaimed** as far as searched. This is **engineering / integration novelty** plus a strong live demo. **[HYPOTHESIS]** (absence of a shipped product; not a proof of absence)

4 What Fault Tolerance Solves (In Plain Words)

4.1 The group-essay analogy

Imagine four friends writing one long essay together, split into parts. Halfway through, one friend's laptop dies and he leaves.

- **Without fault tolerance:** the whole essay is ruined; everyone starts over from the beginning. All prior work is wasted.
- **With fault tolerance:** the group notices he is gone, hands his part to a friend who has a spare copy of the material, and the essay continues. A small hiccup, then back to normal.

Fault tolerance = **the system keeps working even when a part of it breaks**.

4.2 The LLM version

HyperPot splits one model across laptops in an assembly line (worker 1 → worker 2 → worker 3). Because these are real laptops on Wi-Fi, any one can disappear at any moment. With today's tools (llama.cpp RPC) this is fatal: one laptop disconnecting crashes the whole run and the half-written answer is lost ([VERIFIED], §3). HyperPot fixes exactly this: a laptop drops → the controller notices via heartbeat → its share is re-routed to a surviving laptop → the answer keeps streaming.

4.3 A real-time example

Scene. A college lab: student laptops on the lab Wi-Fi together run a Llama model to answer questions for a class. A student asks, "Explain photosynthesis step by step..." and the answer is half-printed on screen. Suddenly one student **grabs their laptop and walks out** — that laptop held one quarter of the model.

- **Today's systems:** the answer freezes and errors out mid-sentence.
- **HyperPot:** a ~1–2 second pause, then the answer continues: "...the plant uses sunlight to turn water and CO_2 into sugar." The user never knew a laptop just left the building.

4.4 The improved recovery idea (capability-aware)

Rather than dumping the dead laptop's whole slice onto one machine, the controller can **check who has spare compute** and split the slice by capability (a strong RTX 4050 takes more layers than a weaker RTX 3050). This is smarter, but note the tradeoff below.

The engineering catch [HYPOTHESIS]. A laptop can only take over a slice if it already holds those model layers in memory. So capability-aware redistribution requires survivors to *pre-load overlapping slices*, which costs memory on every laptop. Two options:

- **Simple (recommended for MVP):** one identical hot-standby holds a copy of a worker's slice → fast, reliable recovery.
- **Smarter (stretch):** several laptops pre-hold overlapping slices → can rebalance by power, but uses more memory and is harder to finish in time.

4.5 What it solves, in one line

> It makes a cluster of unreliable, come-and-go laptops behave like one reliable computer — so people's real laptops dropping off the Wi-Fi does not destroy the work.

5 The Technology Stack

Everything here *extends* the original Go + Rust + llama.cpp design; nothing is rewritten. The one mindset shift: **the controller owns reliability, not llama.cpp**.

5.1 Component by component

Controller — Go KEEP. The coordinator "brain". Handles the HTTP API (user sends prompts), the task queue, worker heartbeats (detect a dead laptop), result collection, and — the novel part — the **failover logic**: "node died → who is alive and has capacity → reassign its layers → re-run the request". Go is well suited to this and the heartbeat/queue pieces already exist.

Worker agent — Rust KEEP. Runs on each laptop. Registers with the controller, sends heartbeats ("alive; here is my CPU/RAM/battery"), loads its assigned slice of the model, runs its layers via llama.cpp, and passes activations to the next worker. Rust compiles to Windows, Linux, and macOS.

Inference engine — llama.cpp KEEP(usage changed).

- **DROP Do NOT use llama.cpp's built-in RPC mode.** It crashes on node loss ([VERIFIED], §3); building on it inherits the exact failure we are beating.
- **KEEP Use llama.cpp as a local engine on each worker**, with the Go controller in charge of coordination and recovery. We route *around* llama.cpp's weak part. This single decision is what makes the project possible.

Model — Meta Llama, small, GGUF KEEP. A small open-weights Llama (e.g. Llama 3.2 1B or 3B) in GGUF, quantized (e.g. Q4). Small = fast load, fits weak laptops, snappy demo. Satisfies the hackathon's "use a Meta model" requirement. **DROP** 70B models — slow demo, and prima.cpp wins that contest anyway.

Networking — TCP/gRPC over Wi-Fi KEEP. Workers ↔ controller over plain TCP/JSON to start (upgrade to gRPC only if needed). Wi-Fi is both the demo environment and the story ("real laptops, real Wi-Fi").

Model split — pipeline (by layers) KEEP. Split by layers (worker 1 = layers 0–8, worker 2 = 9–16, . . .). For recovery, a survivor picks up the dead worker's layers.

Recovery method — request-level re-route KEEP(the core). When a worker dies mid-request, the controller reassigns its layers to a survivor and **re-runs the current request** from the start of generation. The user sees a short pause, then continuation. **DROP** mid-token activation handoff — too hard for the time window.

Code written with — Kiro KEEP. The Go controller, the Rust worker, the failover handling, and the glue are written with Kiro.

5.2 Summary table

Table 2: HyperPot technology stack

Piece	Choice
Controller / failover brain	Go (keep, extend)
Worker agent per laptop	Rust (keep)
Inference engine	llama.cpp — local mode, NOT RPC (keep, change usage)
Model	Meta Llama 3.2 1B/3B, GGUF, quantized (add)
Networking	TCP/JSON then gRPC over Wi-Fi (add)
Model split	by layers (pipeline) (add)
Recovery method	request-level re-route, controller-driven (core)
Code written with	Kiro

6 The Hardware

6.1 The five laptops (verified with the owner)

Table 3: Available demo hardware

#	Laptop	GPU / chip	OS	llama.cpp backend
1	ASUS TUF	RTX 4050	Windows	CUDA
2	ASUS TUF	RTX 4050	Windows	CUDA
3	ASUS ROG	RTX 3050	Linux	CUDA
4	MacBook	Apple M4	macOS	Metal
5	MacBook	Apple M5	macOS	Metal

Five laptops in total. Only three are needed for a convincing "kill one, others survive" demo, so there is a comfortable margin.

6.2 Why this mix helps

- **Two identical TUF 4050s** are the key asset: one can be a **perfect hot standby** for the other. Killing one 4050 and having its identical twin take the exact slice is the cleanest, most reliable failover demo possible.
- **Heterogeneous + cross-OS** (NVIDIA/Windows+Linux vs Apple/macOS) strengthens the story: "different cheap laptops, different chips, different OSes, on Wi-Fi." Note: heterogeneity itself is *not* the novelty (prima.cpp owns that, **[VERIFIED]**); it is a bonus that makes the demo look real.

6.3 The honest warning

Even the "core" three NVIDIA machines are two OSes (2×Windows + 1×Linux), and the Macs add a second backend (Metal). The Rust worker must build on Windows, Linux, and macOS, and llama.cpp needs a CUDA build and a Metal build. This is doable but is where early build time goes. The identical Windows 4050s mean once the worker runs on one, the second is a copy. **[HYPOTHESIS]**

6.4 Recommended configuration

- **Core demo (must work), all-CUDA:** TUF 4050 #1 (worker) + TUF 4050 #2 (worker + hot standby) + ROG 3050 (weaker worker, fewer layers, shows capability-aware assignment). One backend, one build path, lowest risk.
- **Stretch:** add M4 / M5 (Metal) as workers for a cross-OS "pool" finale, and/or run the Go controller on a Mac.
- **Build order rule:** get the all-NVIDIA 3-node failover rock-solid *before* touching CUDA+Metal mixing.

7 How It Works (Architecture)

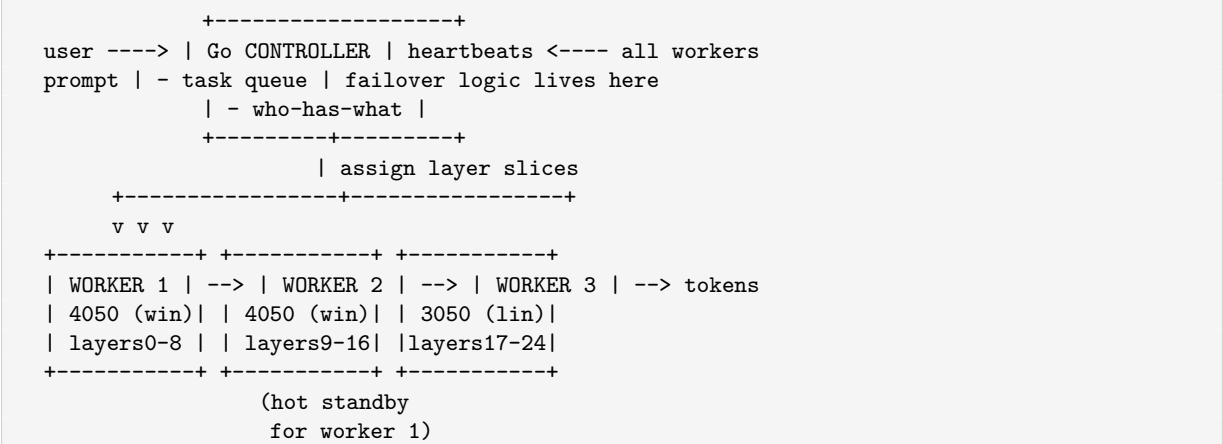
7.1 The normal path

1. The user sends a prompt to the **Go controller** over HTTP.
2. The controller knows which worker holds which layers and sends the prompt down the pipeline.
3. Worker 1 runs its layers, passes activations to worker 2, and so on.
4. The last worker produces tokens, which stream back to the user.
5. Throughout, every worker sends a **heartbeat** to the controller.

7.2 The failover path (the novel core)

1. A worker stops sending heartbeats (lid closed / Wi-Fi dropped / killed).
2. The controller detects the gap within a short timeout.
3. It picks a healthy machine that holds (or can load) the dead worker's layer slice — preferring the identical hot standby.
4. It reassigns the slice and **re-runs the current request**.
5. Token streaming resumes; the user sees a short pause, not a crash.

7.3 Text diagram



```
If WORKER 1 dies: controller detects silence ->
reassigns layers 0-8 to the standby -> re-runs request ->
tokens keep streaming.
```

7.4 Why the controller, not llama.cpp, owns recovery

llama.cpp's RPC mode aborts the process on any peer failure ([**VERIFIED**], §3). By keeping llama.cpp *local* on each worker and putting all coordination in the Go controller, a dead worker becomes a controller event to handle — not a fatal crash of the whole system.

8 The MVP Plan (Phases)

The goal is one thing: **a worker can die mid-generation and the token stream recovers**. Everything is sequenced to de-risk that, lowest-risk first.

Table 4: Build phases

#	Phase	Goal / exit condition
1	One worker works	Rust worker loads a Llama GGUF via llama.cpp, runs its layers, talks to the Go controller. Single-node path correct end to end.
2	Two 4050s, static pipeline	Split a small Llama across the two identical Windows 4050s over Wi-Fi. Correct tokens out. No failover yet.
3	Failover (the core)	Heartbeat detects a dead 4050 → reassign its layers to the twin → re-run request → tokens continue. Rock-solid on the identical pair.
4	Add the Linux ROG 3050	Third worker, weaker, gets fewer layers → shows capability-aware assignment. Still CUDA, low extra risk.
5	(Stretch) add Macs M4/M5	Metal backend; cross-OS, cross-backend pool. Only if phases 1–4 are done and stable.
6	Demo + submission	Script the "kill a laptop" moment; **record a backup video**; write the submission.

8.1 Scope discipline

- **Validate at Phase 1** before investing in failover. If the single-node path is wrong, nothing downstream matters.
- **Macs are optional.** The submission must not depend on the trickiest (CUDA+Metal) integration.
- **Out of scope for MVP:** fine-tuning, security/auth, 70B models, phones, cross-Internet, mid-token handoff. List them as "future work".

8.2 The must-do verification before committing (de-risking)

1. **Kill-test llama.cpp RPC** yourself: confirm it really dies on node loss in your setup (it should, per [**VERIFIED**]). A ∼1-hour test that de-risks the whole premise.
2. **Lock the exact Meta Llama GGUF** so the submission unambiguously "uses a Meta model".
3. **Read the Official Rules** when Meta publishes them (promised before Oct 26) — model-usage requirements can be stricter than the overview.

9 Benchmarks (Measurable & Honest)

Measure the thing that is the differentiator — recovery — and be honest about the thing that is not (raw speed).

9.1 Primary metrics (the differentiator)

Table 5: Primary metrics

Metric	Definition	Target
Recovery time	Wall-clock from node-death to resumed token emission	< 2 seconds
Stream survival	% of generations that complete correctly when 1 of N workers is killed at a random token	∼100% for 1 failure
Output correctness	Tokens identical to a single-node greedy baseline (proves failover does not corrupt output)	exact match

9.2 Secondary metrics (context only — where we do NOT win)

- Tokens/sec and time-per-output-token (TPOT) vs a single-laptop llama.cpp baseline and vs llama.cpp RPC multi-node (the closest apples-to-apples for LAN).
- **Do not stage a speed contest against prima.cpp** — you will lose ([VERIFIED]: prima.cpp reports 5–17× lower TPOT than llama.cpp/exo/dllama). Frame speed as "acceptable overhead for resilience".

9.3 Baselines to run yourself

1. Single-node llama.cpp (the floor).
2. llama.cpp RPC multi-node (to show it crashes on node kill — the before/after contrast that sells the project).
3. HyperPot (same kill, but survives).

Context figures from prima.cpp (674 ms/token for 70B; 26 tok/s for 32B with speculative decoding) are reference only. [VERIFIED]

10 Hackathon Strategy (Meta Global AI Developer Hackathon)

10.1 The verified facts about the event

- [VERIFIED] **Dates:** build Oct 26 – Nov 4 2026; submissions due Nov 4, 9:00 PM ET; judging Nov 4–8; finals Nov 9.
- [VERIFIED] **Track:** Indie developer track — judged on *creativity, technical ambition, and whether it works.*
- [VERIFIED] **Models:** built around Meta Model API (Muse Spark, Muse Image, SAM 3.1), Meta open-weights models, and Muse Code; $150 Meta Model API credits per participant.
- [VERIFIED] **Themes:** "Build for Your World" or "Push the Frontier".

10.2 Why Indie fits HyperPot

The Indie criterion "whether it works" is the purest possible match for a live demo where a laptop is physically killed and the AI keeps talking. Technical ambition is covered by bringing datacenter-grade recovery to cheap laptops.

10.3 The pitch (safe wording)

> "Fault tolerance like this exists in datacenters. It has never run on a pile of cheap student laptops over Wi-Fi with free tools. We made that work — watch, I'll kill one."

This is defensible because every clause is backed by a source (§3). **Never claim** to have invented the algorithm — LUMEN and others got there first ([VERIFIED]); claim that you *brought it down to laptops*, which is true and humble.

10.4 Meta-model requirement

Run a Meta open-weights Llama (small GGUF) through the pipeline. This satisfies "use a Meta model". "Push the Frontier" is the natural theme (a new capability on commodity hardware), though "Build for Your World" works if wrapped around a visible app.

10.5 Demo safety

Always have a **recorded backup video** of the kill-and-recover moment. Never rely solely on live Wi-Fi in front of judges.

11 Risks & Immediate Next Steps

11.1 The honest risks

- **llama.cpp fights failover.** The abort on peer failure is baked into its RPC layer ([VERIFIED]). Mitigation: use llama.cpp *locally* and do recovery in the controller (§7); use request-level re-route, not mid-token handoff.
- **Cross-platform build cost.** Windows + Linux + macOS, CUDA + Metal. Mitigation: all-CUDA core first; Macs are stretch only. [HYPOTHESIS]
- **Novelty challenge from an informed judge.** Mitigation: the "brought it to laptops" framing (§10), never "invented".
- **Live-demo failure.** Mitigation: recorded backup video.

11.2 Guardrails being held

- **KEEP**: Go controller, Rust worker, llama.cpp (local), Meta Llama (small GGUF), Wi-Fi.
- **DROP**: fine-tuning, llama.cpp RPC mode, 70B models, mid-token handoff.
- Win condition: a working live failover demo — not a novelty claim.

11.3 Immediate next steps

1. Kill-test llama.cpp RPC to confirm the premise (§8).
2. Lock the exact Meta Llama GGUF.
3. Read the Official Rules once published.
4. Build Phase 1 (one Rust worker + Go controller + llama.cpp, single node) — only on an explicit "build" instruction.

11.4 Status of this document

This is a planning and research artifact. No HyperPot code has been written yet. The design extends the existing Go + Rust + llama.cpp architecture and is sequenced so the riskiest piece (failover on identical hardware) is proven before any cross-platform work begins.

12 Evidence Log: Kill-Test on Real Hardware

This chapter records the de-risking experiment run *before* any HyperPot code was written, to verify the core premise on the actual demo laptops rather than relying only on documentation.

12.1 Setup

Table 6: Kill-test configuration (26 Oct 2026)

Item	Value
Laptop A (main process)	ROG, RTX 3050 Laptop GPU (3767 MiB), Manjaro Linux, CUDA 13.3
Laptop B (remote node)	TUF, RTX 4050, Windows, llama.cpp b11900 CPU build
Engine	llama.cpp v0.7.0-dev (commit caa8191), built with `-DGGML_CUDA=ON -DGGML_RPC=ON` on the ROG
Model	Llama-3.2-1B-Instruct, Q4_K_M GGUF (771 MB)
Link	Wi-Fi; B exposed `ggml-rpc-server -H 0.0.0.0 -p 50052`
Connectivity	TCP port 50052 reachable from A (verified)

12.2 Run 1 — normal distributed inference (baseline)

A full generation ran across A + B over RPC and **completed successfully**, producing a coherent multi-paragraph story. Final stats: `[ Prompt: 52.7 t/s | Generation: 7.7 t/s ]`, then `Exiting...`. **Conclusion:** distributed inference across two heterogeneous laptops over Wi-Fi *works*. **[VERIFIED]**

12.3 Run 2 — remote node lost (the premise)

A longer run was started. During model data transfer to the remote node, the Wi-Fi link to B dropped packets and the remote node became unreachable. The main process on A did **not** recover — it aborted the entire process:

```
ggml/src/ggml-rpc/ggml-rpc.cpp:685: Remote RPC server crashed
    or returned malformed response
E send failed (bytes_sent=6877944, size_to_send=2559545)
Aborted (core dumped)
```

12.4 What this proves

- **[VERIFIED] llama.cpp RPC has no fault tolerance on this exact hardware.** Loss of the remote node terminates the whole process (`ggml-rpc.cpp:685` → `Aborted`), matching the documentation (§3).
- **[VERIFIED] The failure is realistic, not contrived.** It was triggered by ordinary Wi-Fi instability — no deliberate kill was even needed. This is precisely the scenario HyperPot targets.
- **Answer to "does it restart from the beginning?"** No — plain llama.cpp does not restart at all; it crashes and must be re-launched by hand. HyperPot's contribution is to make the controller detect the loss, re-route the work, and **automatically re-run the request** so generation continues.

12.5 Note for the live demo

The crash here came partly from random Wi-Fi drops during a large transfer to a CPU-only node. For a reproducible stage demo, the "kill" should be a **deliberate, clean action** (Ctrl+C

on the remote server, or lid-close) rather than relying on chance packet loss. The before/after contrast — llama.cpp aborts vs HyperPot recovers — is the core of the demonstration. Both log files (`killtest.log`, `killtest2.log`) are retained as evidence.

13 References & Sources

Every source below was consulted directly during the research for this document. Each is tagged by how it was used. Access dates: October 2026.

13.1 Research papers (competitive landscape)

[P1] **Prima.cpp: Fast 30–70B LLM Inference on Heterogeneous and Low-Resource Home Clusters.** Z. Li et al., arXiv:2504.08791 (v1 Apr 2025, v3 Jul 2026). https://arxiv.org/abs/2504.08791.
Used for: heterogeneous/Wi-Fi home-cluster inference; Halda scheduler; the 5–17× lower TPOT claim vs llama.cpp/exo/dllama; static scheduling (no mid-inference recovery). **[VERIFIED]**

[P2] **Petals: Collaborative Inference and Fine-tuning of Large Models.** A. Borzunov et al., arXiv:2209.01188. https://arxiv.org/abs/2209.01188.
Used for: BitTorrent-style swarm; exposing hidden states for fine-tuning. **[VERIFIED]**

[P3] **Distributed Inference and Fine-tuning of Large Language Models Over The Internet.** A. Borzunov et al., arXiv:2312.08361.
https://arxiv.org/abs/2312.08361.
Used for: churn-tolerant inference over the Internet for 50B+ models ("any device can disconnect abruptly"). **[VERIFIED]**

[P4] **Distributed Llama — Distributed Inference of Large Language Models with Slow Synchronization over Ethernet.** B. Tadych, 2024.
https://github.com/b4rtaz/distributed-llama (report.pdf).
Used for: tensor-parallel home-device inference; synchronization cost. **[VERIFIED]**

[P5] **LUMEN: Coordinated Failure Recovery for Distributed LLM Serving.** Z. Cao et al., arXiv:2606.17787. https://arxiv.org/abs/2606.17787.
Used for: proof that load-aware, interrupted-request recovery *already exists* in research (datacenter/GPU). Key to the honest-novelty statement. **[VERIFIED]**

[P6] **Intelligent Orchestration of Distributed Large Foundation Model Inference at the Edge.** arXiv:2504.03668.
https://arxiv.org/abs/2504.03668.
Used for: capacity-aware workload distribution + dynamic partition migration + real-time re-split (the capability-aware recovery idea, already published). **[VERIFIED]**

[P7] **FailSafe: High-performance Resilient Serving.** arXiv:2511.14116. https://arxiv.org/abs/2511.14116.
Used for: serving that survives multiple GPU failures. **[VERIFIED]**

[P8] **PipeLive: Efficient Live In-place Pipeline Parallelism Reconfiguration for Dynamic LLM Serving.** arXiv:2604.12171.
https://arxiv.org/abs/2604.12171.
Used for: live pipeline reconfiguration during serving. **[VERIFIED]**

[P9] **DynaPipe: Dynamic Layer Redistribution for Pipeline-Parallel LLM Inference.** NeurIPS 2025.
Used for: dynamic layer redistribution + async KV-cache migration during inference. **[VERIFIED]**

[P10] **EnerInfer: Energy-Aware On-Device LLM Inference.** arXiv:2606.23001. https://arxiv.org/abs/2606.23001.
Used for: evidence that battery/thermal-aware on-device LLM scheduling *already exists* — which ruled out that angle as a novelty. **[VERIFIED]**

[P11] **LLM Inference at the Edge: Mobile, NPU, and GPU Performance Efficiency Trade-offs Under Sustained Load.** arXiv:2603.23640.
Used for: thermal throttling as the dominant constraint on-device. **[VERIFIED]**

[P12] **MNN-AECS: Energy Optimization for LLM Decoding on Mobile Devices via Adaptive Core Selection.** arXiv:2506.19884.
Used for: further prior art on energy-adaptive on-device decoding. **[VERIFIED]**

13.2 Official repositories & documentation

[R1] **llama.cpp — RPC tool README.** ggml-org/llama.cpp.
https://github.com/ggml-org/llama.cpp/blob/master/tools/rpc/README.md.
Used for: the RPC backend design and its insecurity warning. **[VERIFIED]**

[R2] **llama.cpp RPC — peer-failure behaviour.** LlamaCppEx binding docs. https://llama-cpp-ex.hexdocs.pm/LlamaCppEx.RPC.html.
Used for: the verified statement that a peer crash/drop terminates the process (`RPC_STATUS_ASSERT`/`GGML_ABORT`). **[VERIFIED]**

[R3] **llama.cpp issue #11508** — crash while loading in multi-node RPC after a node crash. https://github.com/ggml-org/llama.cpp/issues/11508. **[VERIFIED]**

[R4] **llama.cpp issue #9214** — RPC backend crash.
https://github.com/ggml-org/llama.cpp/issues/9214. **[VERIFIED]**

[R5] **Exo.** exo-explore/exo. https://github.com/exo-explore/exo.
Used for: features (auto-discovery, MLX, Thunderbolt RDMA, tensor parallel); open issues on instability (#1726, #1847, #2007, #775). **[VERIFIED]**

[R6] **Deep Dive: Exo — Distributed AI Inference on Consumer Hardware** (review, Sept 2025). *Used for:* "critical gaps in security, fault tolerance, and operational tooling". **[VERIFIED]**

[R7] **prima.cpp repository.** https://github.com/OpenCPIL/prima.cpp (also gitee.com/zonghang-li/prima.cpp). **[VERIFIED]**

[R8] **QVAC Fabric LLM** (Tether Data).
https://qvac.tether.io/products/fabric and the Dec 2025 launch announcement on `tether.io`.
Used for: existence of an edge-first inference + LoRA fine-tuning framework. Existence **[VERIFIED]**; technical internals **[UNKNOWN]**.

13.3 Hackathon (rules & requirements)

[H1] **Meta Global AI Developer Hackathon — official page.**
https://dev.meta.ai/events/global-hackathon.
Used for: dates, Indie vs Startup tracks and their judging criteria, Meta Model API / open-weights model usage, $150 credits, themes, eligibility. **[VERIFIED]**

[H2] **Cerebral Valley event listing.**
https://cerebralvalley.ai/e/meta-ai-developer-hackathon.
Used for: the 20,000-applicant cap and rolling evaluation. **[VERIFIED]**

13.4 Evidence-strength note

Items marked **[VERIFIED]** are quoted or closely paraphrased from the named source. Where a source is marketing material (e.g. QVAC Fabric), only the *existence* of the capability is treated as verified; its internal mechanisms remain **[UNKNOWN]** until primary technical documentation is available. No claim of HyperPot's novelty rests on an **[UNKNOWN]** item.